

DAA - labo2

DAA_B_9

Bleuer Rémy, Changanagui Yoann, Rajadurai Thirusan

03.10.2025

Introduction

Ce laboratoire a pour but de nous familiariser avec les bases du développement Android en utilisant Kotlin. Ce labo vise particulièrement les Activités et les Fragments. Aux travers de trois manipulations différentes, nous entraînons la navigation entre les activités, les cycles de vie d'une activité, l'utilisation et la gestion de fragments. Le but final de ce labo est de nous faire comprendre comment Android gère ces différents composants, afin d'utiliser les meilleures pratiques pour s'assurer d'avoir une application robuste et réactive.

Exercice 1 : Les Activités

Choix d'implémentations

Il y a deux activités distinctes, une première `MainActivity` qui affiche le message de bienvenue, et `EditUsernameActivity` qui permet de choisir un nom et de l'envoyer à la première.

Utilisation des contrats de la librairie `AndroidX` avec `registerForActivityResult` comme vu lors de la théorie.

Utilisation de `Log.d` dans chaque méthode pour logger les différents chemins parcourus.

Utilisation d'un `ConstrainLayout` afin de pouvoir centrer horizontalement et verticalement dans le parent.

Questions

- Que se passe-t-il si l'utilisateur appuie sur « back » lorsqu'il se trouve sur la seconde Activité ?

Il revient à la première activité sans modification du texte. Ceci dans les deux cas, que ça soit le message de bienvenue par défaut ou le message modifié avec un nouveau nom.

- Veuillez réaliser un diagramme des changements d'état des deux Activités pour les utilisations suivantes, vous mettrez en évidence les différentes instances de chaque Activité :

J'ai mal lu l'énoncé et ai commencé par des diagrammes de séquence, ils sont à dispositions dans le dossier images

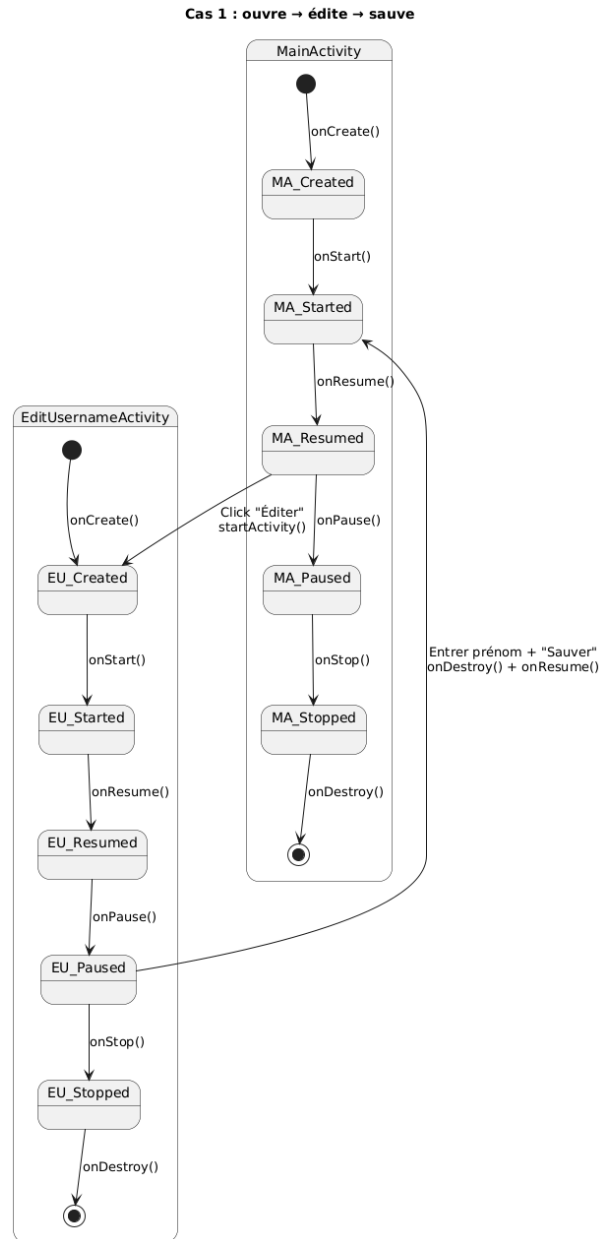


Figure 1: L'utilisateur ouvre l'application, clique sur le bouton éditer, renseigne son prénom et sauve.

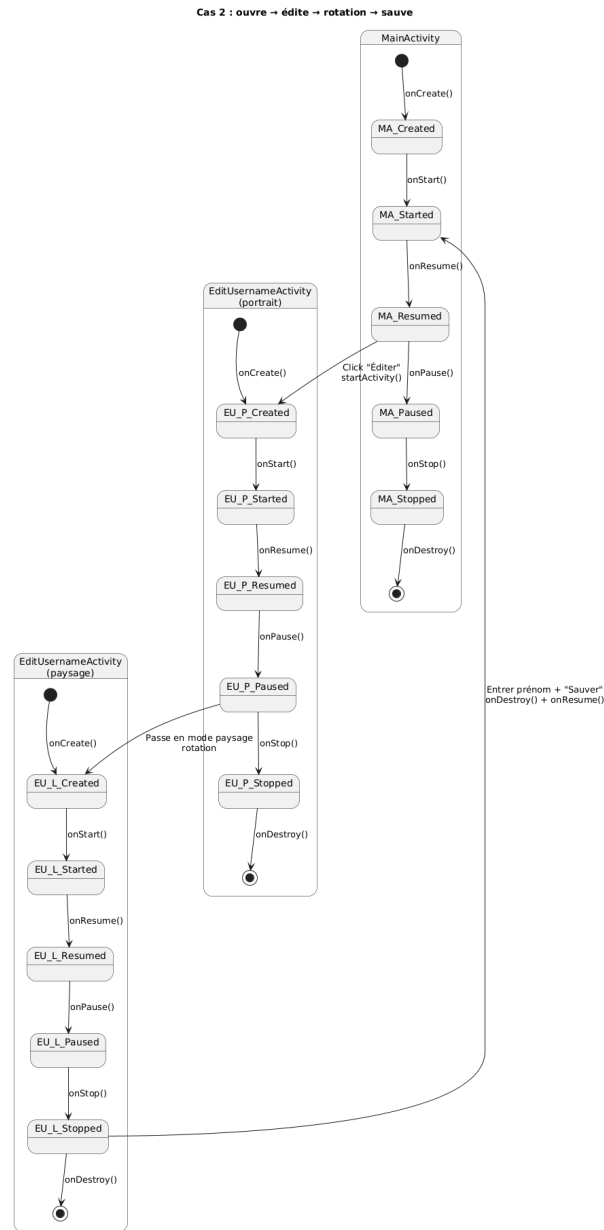


Figure 2: L'utilisateur ouvre l'application en mode portrait, clique sur le bouton éditer, bascule en mode paysage, renseigne son prénom et sauve.

- Que faut-il mettre en place pour que vos Activités supportent la rotation de l'écran ? Est-ce nécessaire de le réaliser pour les deux Activités, quelle est la différence ?

Il faut sauvegarder l'état de l'activité avant la rotation et le restaurer après. Seulement dans `MainActivity` car c'est la seule activité qui a un état à conserver (le nom). Dans `EditUsernameActivity`, le champ de texte restaure son contenu automatiquement.

Tests

Test effectué	Résultat attendu	Résultat obtenu
Ouvrir l'application	Affichage du "Bonjour"	OK
Cliquer sur le bouton éditer	Affichage d'un champ de text avec un bouton sauver	OK
Cliquer sur le bouton sauver avec un nom	Affichage du "Bonjour + nom"	OK
Cliquer sur le bouton sauver avec champ vide	Il ne devrait pas avoir de modification du message précédent	KO le nom est vide
Rotation de l'écran	Les texts et les boutons s'adaptent à l'écran	OK
Cliquer sur le bouton "back" depuis la 2ème activité	Retour à la 1ère activité sans changement de text	OK

Exercice 2 : Les Fragments, premiers pas

Choix d'implémentation

Le layout utilise un `ConstraintLayout` avec les éléments suivants :

1. **AppBar** : En haut de l'écran
2. **Container pour CounterFragment** : Occupe 50% de l'espace disponible sous l'AppBar
3. **Container pour ColorFragment** : Occupe les 50% restants

Choix techniques :

- Utilisation de `FrameLayout` comme conteneurs de fragments (pratique standard Android)
- Division de l'écran en deux parties égales via des contraintes :
 - `layout_height="0dp"` avec contraintes top/bottom
 - `layout_constraintHeight_percent="0.5"` pour 50% de hauteur chacun
- L'AppBar est placée en haut et ne compte pas dans la division 50/50

Questions

- Les deux Fragments fournis implémentent la restauration de leur état. Si on enlève la sauvegarde de l'état sur le `ColorFragment` sa couleur sera tout de même restaurée, comment pouvons-nous expliquer cela ?

Si on enlève la sauvegarde de l'état sur le `ColorFragment`:

Normalement les états des vues ne sont pas enregistrés par défaut SAUF pour les champs avec un ID. Ici les champs avec ID sont enregistrés et suffisent à recalculer (via les listeners) la couleur, car techniquement seulement les états des "curseurs" du fragment sont enregistrés et partir de cela le fragment va redéterminer la couleur (qui était donc là dans l'état précédent).

- Si nous plaçons deux fois le `CounterFragment` dans l'Activité, nous aurons deux instances indépendantes de celui-ci. Comment est-ce que la restauration de l'état se passe en cas de rotation de l'écran ?

Malgré la duplication du fragment les 2 différent conteneurs de fragments. Donc à l'appel de ces conteneurs via la classe `R`, chacun aura une identification unique -> `R.id...` par container, qui auront chacun leur propriétés. Il n'y aura donc aucune différence malgré la répétition du fragment.

Tests

Test effectué	Résultat attendu	Résultat obtenu
Ouvrir l'application	Affichage des deux fragments (CounterFragment et ColorFragment)	OK
Incrémenter le compteur	Le compteur augmente de 1	OK
Rotation de l'écran	Les états des fragments sont restaurés	OK
Modifier la couleur	La couleur de fond change selon les curseurs	OK
Rotation après modification couleur	La couleur est restaurée correctement	OK

Exercice 3 : Le FragmentManager

Questions

- A l'initialisation de l'Activité, comment peut-on faire en sorte que la première étape s'affiche automatiquement ?

```
<androidx.fragment.app.FragmentContainerView
android:id="@+id/fragment_container"
android:layout_width="0dp"
android:layout_height="0dp"
android:name="ch.heigvd.iict.daa.template.Step" <!-- ici -->
app:layout_constraintTop_toBottomOf="@id/appbarlayout"
app:layout_constraintBottom_toTopOf="@id/buttons"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent">
</androidx.fragment.app.FragmentContainerView>
```

Dans le code du layout (*donc le xml*) on ajoute la **référence** de la classe de notre fragment dans le composant **FragmentContainerView** qui devra être instancié lors de l'affichage de la vue de l'Activité (*OnCreate*).

se référer au support de cours “3 - Les briques de base - Activités, Fragments et Services” Slide 41

- Comment pouvez-vous faire en sorte que votre implémentation supporte la rotation de l'écran ? Nous nous intéressons en particulier au maintien de l'état de la pile de Fragments et de l'étape en cours lors de la rotation.

En effet, lorsqu'il y a une rotation de l'écran (*passage au mode “landscape”*), la pile de Fragments est recrée à l'identique par le **FragmentManager**, mais pas l'**état interne** des Fragments.

se référer au support de cours “3 - Les briques de base - Activités, Fragments et Services” Slide 45

Autrement dit lorsque l'écran pivote, mon activité **MainActivityFragment2.kt** est détruite, on passe à celle dédié pour le mode landscape. Donc la variable **id** n'existe plus. Une nouvelle instance de l'activité est créée, **onCreate()** est appelé, et **id** est réinitialisé à 0. Même si mon **FragmentManager** peut restaurer la pile de fragments mon compteur **id** lui est perdu.

La solution consiste à **sauvegarder** l'id lorsque l'activité est sur le point d'être détruite grâce à la méthode à la méthode **onSaveInstanceState(bundle)**. Puis de la restaurer lorsque la nouvelle instance est créée, **onViewStateRestored(bundle)**.

La documentation officielle

- Dans une transaction sur le Fragment, quelle est la différence entre les méthodes **add** et **replace** ?

La méthode **add()** ajoute un nouveau fragment au-dessus de ce qui existe déjà dans le conteneur. Si un fragment est déjà présent dans le conteneur, **add()** place le nouveau fragment par-dessus l'ancien. On le remarque avec notre exercice car les *ids* affichés se superposent. Les deux fragments sont techniquement dans le conteneur en même temps. Le fragment précédent n'est pas détruit. Son cycle de vie continue (*pas de onDestroy()*), mais il passe à l'état **onStop()**.

La méthode **replace()** remplace le fragment actuellement dans le conteneur par un nouveau. C'est l'équivalent d'un appel à **remove()** pour l'ancien fragment, suivi d'un appel à **add()** pour le nouveau. Le fragment existant est complètement retiré de la vue avant que le nouveau ne soit ajouté. À tout moment, il n'y a qu'un **seul fragment** géré activement dans le conteneur.

A noter que je sauvegarde les fragments dans une “backstack” géré par le `FragmentManager` (*ou `FragmentManager`*) avec l’appel à `.addToBackStack(null)`

Voir plus dans la documentation officielle

Choix d’implémentations

En ce qui concerne l’implémentation, elle a majoritairement été discuté lors de la section questions-réponses.

Néanmoins il est intéressant de rappeler l’architecture employé pour la réalisation de cette partie :

```
app/
| - kotlin+java/
|   | - ch.heigvd.iict.daa.template/
|       | - MainActivityFragment2.kt
|       | - Step.kt
| - res/
|   | - layout
|       | - fragmentmanager_activity.xml
|       | - fragment_step.xml
```

On retrouve les éléments tel que, `fragmentmanager_activity.xml` qui est le layout qui contient lui même un `FragmentManager` qui contiendra au lancement de l’application mon second layout `fragment_step.xml` (*la vue du fragment/design*).

`MainActivityFragment2.kt` est le programme qui lance l’activité (*on en a qu’une seule pour cette exercice*). C’est-elle qui gère le **FragmentManager** donc tout ce qui concerne la pile de fragments et d’états. Ainsi que `Step.kt` qui permet de manipuler le fragment.

Tests

Test effectué	Résultat attendu	Résultat obtenu
Ouvrir l’application	Affichage d’un fragment par défaut	OK
Appuyer sur le bouton Back sans ajouté de fragments	rien	OK
Appuyer sur le bouton Close	Ferme l’activité (<i>application</i>)	OK
Appuyer sur le bouton Next	Génère un nouveau fragment avec l’id suivant	OK
Appuyer sur le bouton Back après ajout de fragments	Revient au fragment précédent et retire l’ancien	OK
Rotation de l’écran	Etat de la pile ainsi que l’état interne des fragments sauvegardé	OK

Conclusion

Ce labo nous a permis de comprendre le fonctionnement des briques principales d’Android qui sont les activités et les fragments.

La première partie nous a appris à gérer la navigation et la communication entre activités, ainsi qu’à observer différents événements comme la rotation de l’écran ou les cycles de vie.

La deuxième partie nous a introduit aux fragments en nous montrant comment organiser une activité en plusieurs composant indépendants.

Finalement, la troisième partie nous a montré le `FragmentManager` et la gestion de pile de fragments.

Grâce à ce labo, nous savons maintenant comment fonctionner pour avoir une application stable et fluide en utilisant ces différents composants Android.

Note sur l'utilisation de l'IA

Nous avons utilisé GitHub Copilot pour nous aider à compléter les commentaires des fichiers Kotlin, en respectant le format KDoc. Nous avons également utilisé ChatGPT pour reformuler certaines phrases du rapport, afin d'améliorer la clarté du texte.